

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

PRÁCTICA EXPERIMENTAL UNIDAD III

MATERIA:

INGENIERÍA DE REQUERIMIENTOS

DOCENTE:

DR. GUERRERO ULLOA GLEISTON CICERÓN

INTEGRANTES:

FUERTES ARRAES EDSON DANIEL, CI: 0929115087, Correo: efuertes2@uteq.edu.ec

VITERI GARCIA JONATHAN ENRIQUE, CI: 1250424734, Correo: Jonathanvg2407@gmail.com

CONTRERAS CHAVEZ KEVIN GERMAN, CI: 1251167720, Correo: kcontrerasc3@uteq.edu.ec

CURSO:

CUARTO SEMESTRE PARALELO "B"

Repositorio GitHub:

https://github.com/MiloSaurio4kHD/PE3_U3_FUERTES_VITERI_CONTRERAS_MUNDIPETS.git

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1. Fundamento teórico	4
1.1. Marco normativo aplicado a la especificación de requisitos	4
1.2. Documentación de requisitos	4
1.3. Documento ERS/SRS según IEEE/ISO/IEC 29148:2018	4
1.4. Modelos de datos (perspectiva de datos)	4
1.5. Modelos funcionales (perspectiva funcional)	4
1.5.1. Diagramas de Flujo de Datos, DFD	4
1.5.2. Elementos de un DFD	4
1.5.3. Niveles de DFD y balanceo	5
1.5.4. Diagrama de actividades UML	5
1.5.5. Aplicación a MundiPets	5
1.6. Modelos de comportamiento (perspectiva de comportamiento)	6
1.6.1. Statecharts	6
1.6.2. Elementos de los diagramas de estado	6
1.6.3. Diagramas de estado UML	7
1.6.4. Aplicación a MundiPets	7
1.7. Interrelaciones entre las tres perspectivas	7
1.7.1. Relación entre perspectiva de datos y perspectiva funcional	7
1.7.2. Relación entre perspectiva funcional y perspectiva de comportamiento	8
1.7.3. Relación entre perspectiva de datos y perspectiva de comportamiento	8
1.7.4. Aplicación a MundiPets e importancia de la integración	8
2. Revisión y refinamiento del SRS parcial	9
2.1. Registro de defectos del SRS	9
2.2. Corrección con sintaxis EARS	9
2.3. Tabla de atributos completa (RF y RNF)	9
3. Modelo de datos: ER y clases UML	9
3.1. Identificación de entidades y relaciones	9
3.2. Diagrama Entidad-Relación (Crow's Foot)	9
3.3. Diagrama de clases UML	9
4. Modelo funcional: DFD y actividades UML	10
4.1. Diagrama de contexto (DFD Nivel 0)	10
4.2. DFD Nivel 1	11
4.3. DFD esencial	12
4.4. Diagrama de actividades UML	13
5. Modelo de comportamiento: estados y secuencia	14
5.1. Diagrama de máquina de estados UML	14
5.2. Diagrama de secuencia UML	14
6. Matriz de trazabilidad y tabla comparativa	14
6.1. Matriz de trazabilidad	14
6.2. Tabla comparativa de los tres modelos	15
6.3. Conclusiones	16
Anexo A – Tabla de atributos completa de todos los RF y RNF	17
Anexo B – Registro de defectos del SRS con correcciones aplicadas	17
Anexo C – Exportaciones originales de todos los diagramas (alta resolución)	17
Anexo D – Referencias IEEE y declaración de uso de IA	17

Índice de figuras

1.	Diagrama de contexto (DFD Nivel 0) del sistema MundiPets.	10
2.	Diagrama de Flujo de Datos de Nivel 1 del sistema MundiPets.	11
3.	Diagrama de Flujo de Datos Esencial del sistema MundiPets.	12
4.	Diagrama de actividades UML del proceso principal en MundiPets.	13

Índice de tablas

1.	Registro de defectos del SRS parcial.	9
2.	Matriz de trazabilidad integral de componentes del sistema MundiPets.	14
3.	Cuadro comparativo de los modelos de análisis aplicados en MundiPets.	15

1 Fundamento teórico

1.1 Marco normativo aplicado a la especificación de requisitos

1.2 Documentación de requisitos

1.3 Documento ERS/SRS según IEEE/ISO/IEC 29148:2018

1.4 Modelos de datos (perspectiva de datos)

1.5 Modelos funcionales (perspectiva funcional)

La perspectiva funcional en la ingeniería de requisitos permite documentar qué información es recibida desde el contexto del sistema, cómo es manipulada por el sistema o por una de sus funciones específicas, y qué datos son devueltos nuevamente hacia el entorno [1]. Esta perspectiva es importante porque permite representar la transformación de los datos de entrada en datos de salida, mostrando cómo el sistema procesa la información necesaria para cumplir con sus objetivos [1]. Además, no solo se enfoca en los datos que entran y salen, sino también en el orden en el que las funciones deben ejecutarse para procesar correctamente dicha información [1]. Por esta razón, los modelos funcionales ayudan a comprender las funcionalidades que el sistema debe implementar y permiten representar de forma clara los procesos principales que responden a las necesidades de los interesados.

Desde esta perspectiva, el sistema puede analizarse como un conjunto de funciones que reciben información, la procesan y generan resultados útiles para los actores externos. En el caso de un sistema de software, esto permite identificar qué procesos deben existir, qué información necesita cada proceso para ejecutarse y qué resultados debe entregar. Por ello, la perspectiva funcional es especialmente útil en la documentación de requisitos, ya que ayuda a pasar de necesidades generales de los usuarios a procesos más concretos que pueden ser modelados y posteriormente implementados [1].

1.5.1 Diagramas de Flujo de Datos, DFD

Uno de los modelos más utilizados dentro de la perspectiva funcional es el Diagrama de Flujo de Datos, conocido como DFD. Este tipo de diagrama permite representar la funcionalidad de un sistema mediante procesos, flujos de datos, almacenes de datos y elementos externos que interactúan con el sistema [1]. Los DFD permiten modelar el sistema en diferentes niveles de abstracción, lo que facilita analizarlo primero de manera general y luego descomponerlo en procesos más detallados [1]. De esta forma, el DFD ayuda a representar cómo la información circula dentro del sistema, cómo se transforma y cómo se relaciona con los actores o sistemas externos.

Además, los DFD son útiles porque permiten especificar, construir y visualizar el modelo de un sistema mediante una vista gráfica [2]. Esto facilita que tanto analistas como usuarios comprendan cómo se mueven los datos dentro de un sistema de información [2]. También permiten definir requisitos funcionales de una manera más clara, debido a que muestran qué procesos se ejecutan, qué entradas reciben, qué salidas generan y qué datos deben almacenarse o consultarse durante el funcionamiento del sistema [2]. Por ello, los DFD son una herramienta adecuada para representar procesos de negocio y requisitos funcionales en sistemas grandes o complejos.

1.5.2 Elementos de un DFD

Un DFD está compuesto por cuatro elementos principales: procesos, flujos de datos, almacenes de datos y entidades externas [2].

El proceso representa una actividad o función realizada por una razón de negocio específica [2]. Desde la perspectiva funcional, un proceso consume datos de entrada, los transforma y produce datos de salida como resultado [1]. Por ejemplo, en un sistema, un proceso puede encargarse de registrar información, validar datos, generar una solicitud o emitir una notificación. Lo importante es que cada proceso represente una función clara del sistema y tenga relación con la transformación de información.

El flujo de datos representa la información que se transporta entre procesos, almacenes de datos y entidades externas [1, 2]. Este flujo puede representar datos simples o un conjunto lógico de información que se mueve dentro del sistema [2]. Por ejemplo, una solicitud, un perfil de mascota, datos médicos o una notificación pueden representarse como flujos de datos si son enviados de un elemento a otro. Los flujos permiten entender qué información necesita cada proceso y qué información produce luego de ejecutarse.

El almacén de datos representa información persistente que el sistema conserva para ser consultada o modificada por los procesos [1]. En un DFD, los procesos pueden leer información desde un almacén o escribir nuevos datos

en él [1]. Por ejemplo, en un sistema como MundiPets podrían existir almacenes como “Datos de mascotas”, “Solicitudes de adopción”, “Usuarios registrados” o “Historial médico”. Estos almacenes permiten representar la información que no desaparece al terminar un proceso, sino que queda guardada para ser utilizada posteriormente.

Finalmente, la entidad externa representa una persona, organización o sistema que se encuentra fuera del sistema, pero que interactúa con él mediante el intercambio de información [1, 2]. Las entidades externas pueden enviar datos al sistema o recibir información generada por este. Sin embargo, no forman parte del funcionamiento interno del sistema ni pueden ser modificadas por el proceso de desarrollo [1]. En MundiPets, algunas entidades externas podrían ser el propietario, el adoptante, el refugio, la veterinaria o el administrador, dependiendo del proceso que se esté modelando.

1.5.3 Niveles de DFD y balanceo

Los DFD se organizan en niveles de abstracción para manejar mejor la complejidad del sistema [1]. El nivel más general es el diagrama de contexto, que representa el sistema como un único proceso central conectado con las entidades externas y los principales flujos de información [2]. Este diagrama permite definir los límites del sistema y mostrar de forma general qué actores o sistemas interactúan con él. Normalmente, en el diagrama de contexto no se muestran almacenes de datos, porque su objetivo principal es presentar el alcance general del sistema y sus relaciones externas [2].

Después del diagrama de contexto se construye el DFD de nivel 0, que descompone el proceso general en los procesos principales del sistema [2]. En este nivel ya se pueden representar procesos internos importantes, almacenes de datos y flujos más detallados. Luego, cada proceso del nivel 0 puede descomponerse en un DFD de nivel 1, donde se explica con mayor detalle cómo funciona internamente un proceso específico [2]. Esta descomposición permite analizar el sistema paso a paso, desde una vista general hasta una vista más detallada de sus funciones.

Un principio importante en los DFD es el balanceo. El balanceo consiste en asegurar que los flujos de datos, almacenes y entidades externas que aparecen en un nivel superior estén correctamente representados en el nivel inferior que lo descompone [2]. Esto evita inconsistencias entre diagramas y permite mantener coherencia cuando un proceso se detalla en niveles más bajos. Por ejemplo, si en el diagrama de contexto aparece una veterinaria enviando información médica al sistema, esa interacción debe conservarse de forma coherente en el DFD nivel 0 o en el nivel donde se detalle el proceso correspondiente [2].

1.5.4 Diagrama de actividades UML

Además de los DFD, la perspectiva funcional también puede complementarse con diagramas de actividades UML. Estos diagramas son adecuados para modelar secuencias de acciones y flujos de control entre actividades [1]. A diferencia del DFD, que se concentra principalmente en el movimiento y transformación de datos, el diagrama de actividades permite representar el orden en el que se ejecutan las acciones dentro de un proceso [1]. Por ello, resulta útil para describir procesos de negocio, casos de uso o procedimientos donde se necesita mostrar qué actividad ocurre primero, qué condiciones cambian el flujo y qué actores participan.

Un diagrama de actividades UML está compuesto por acciones, flujos de control, nodos de inicio, nodos de fin, nodos de decisión, barras de sincronización, flujos de objeto y particiones de actividad [1]. Las acciones representan tareas concretas que se ejecutan dentro del proceso. El flujo de control se representa mediante flechas que indican el paso de una acción a otra. Los nodos de decisión permiten dividir el flujo según condiciones, por ejemplo, si una solicitud cumple o no cumple ciertos requisitos. Las barras de sincronización permiten representar concurrencia mediante fork y join, donde un flujo puede dividirse en actividades paralelas o unirse nuevamente antes de continuar [1]. Los flujos de objeto permiten mostrar cómo ciertos datos u objetos pasan entre acciones dentro del proceso [1]. Esto es útil cuando se necesita representar que una actividad produce un documento, una solicitud, una validación o cualquier información necesaria para la siguiente actividad. Además, las swimlanes o particiones de actividad permiten asignar responsabilidades a los actores o componentes participantes [1]. De esta manera, se puede observar qué acciones corresponden al usuario, cuáles al sistema, cuáles al refugio y cuáles a la veterinaria. Esto ayuda a organizar mejor el proceso y a identificar responsabilidades dentro del flujo funcional.

1.5.5 Aplicación a MundiPets

En el sistema MundiPets, los modelos funcionales pueden aplicarse al proceso de gestionar adopción o cruzar responsable. Desde la perspectiva funcional, el sistema recibe información de mascotas, datos médicos, datos de usuarios, solicitudes de adopción o datos de compatibilidad, y transforma esa información en resultados como publicaciones, solicitudes registradas, validaciones, recomendaciones o notificaciones [1]. Este análisis permite identificar las funciones principales que el sistema debe realizar, como registrar mascota, publicar perfil,

consultar información, validar datos médicos, gestionar solicitud de adopción, evaluar compatibilidad y notificar resultados.

En un DFD de MundiPets, los propietarios, refugios, usuarios interesados, veterinarias y administradores pueden representarse como entidades externas que intercambian información con el sistema [2]. Los procesos internos pueden representar funciones como “Registrar mascota”, “Validar información médica”, “Gestionar solicitud de adopción”, “Publicar mascota” o “Evaluar compatibilidad para cruce”. Los almacenes de datos pueden representar información persistente como “Mascotas”, “Usuarios”, “Solicitudes”, “Historial médico” o “Publicaciones”. Los flujos de datos permiten representar la información que circula entre estos elementos, como datos de mascota, solicitud de adopción, documentos médicos, estado de validación o notificación de respuesta [1, 2].

El uso de niveles de DFD también permite organizar el análisis de MundiPets. En el diagrama de contexto, el sistema puede mostrarse como un único proceso central que interactúa con propietarios, adoptantes, refugios, veterinarias y administradores [2]. En el DFD nivel 0, ese proceso general puede dividirse en procesos principales como gestión de usuarios, gestión de mascotas, gestión de adopciones, validación veterinaria y comunicación entre usuarios. Luego, en un DFD nivel 1, se puede detallar un proceso específico, como gestionar adopción, mostrando cómo se registra la solicitud, cómo se revisa la información médica, cómo el propietario o refugio toma una decisión y cómo el sistema notifica el resultado. En todos estos niveles debe mantenerse el balanceo para asegurar que los flujos y entidades externas se conserven de forma coherente [2].

Finalmente, el diagrama de actividades UML puede utilizarse para representar paso a paso el flujo de adopción o cruce responsable en MundiPets [1]. Por ejemplo, el flujo puede iniciar cuando un usuario consulta una mascota disponible, revisa su perfil, envía una solicitud, el sistema registra la solicitud, el propietario o refugio la revisa, se verifica la información médica y finalmente se acepta o rechaza el proceso. Las swimlanes permitirían separar las responsabilidades del usuario, del sistema, del propietario o refugio y de la veterinaria [1]. Los nodos de decisión ayudaron a representar condiciones como “¿cumple requisitos de adopción?” o “¿información médica validada?”, mientras que los flujos de objeto podrían representar la solicitud, el perfil médico o la respuesta enviada al usuario. De esta manera, los modelos funcionales permiten comprender de forma organizada cómo MundiPets transforma la información y ejecuta sus procesos principales.

1.6 Modelos de comportamiento (perspectiva de comportamiento)

La perspectiva de comportamiento se centra en documentar los requisitos de un sistema de una manera orientada a los estados [2]. En este enfoque, el sistema se describe a partir de los estados internos en los que puede encontrarse, los eventos que ocurren en su contexto y las reacciones que estos provocan [2]. Es fundamental para definir no solo cómo cambia el sistema internamente, sino también las condiciones específicas que deben cumplirse para realizar una transición entre estados y los efectos o acciones que el sistema produce en su entorno como resultado de dichos cambios [2].

Los modelos de comportamiento sirven para capturar y representar de forma precisa el carácter dinámico, reactivo o impulsado por eventos de un sistema de información [2]. A diferencia de los modelos funcionales estáticos, estas representaciones permiten visualizar cómo el software “vive” evoluciona en el tiempo en respuesta a estímulos externos. Son herramientas esenciales cuando la complejidad del sistema exige detallar su naturaleza reactiva frente a situaciones particulares del negocio [2].

1.6.1 Statecharts

Los statecharts son una técnica avanzada de modelado utilizada para describir el comportamiento reactivo de sistemas complejos, extendiendo la teoría de los autómatas de estados finitos tradicionales [2]. Su propósito principal es permitir la abstracción y el manejo de la complejidad mediante conceptos de jerarquización y concurrencia [2]. En estos diagramas, se pueden agrupar estados relacionados, definir transiciones protegidas por condiciones lógicas y modelar múltiples procesos que ocurren de manera simultánea e independiente dentro de una misma entidad [2].

1.6.2 Elementos de los diagramas de estado

Un diagrama de estado se integra por varios elementos que estructuran el flujo conductual. El estado representa un periodo de tiempo donde el sistema exhibe una conducta particular mientras espera un evento para realizar una transición [2]. Esta transición puede estar sujeta a una condición lógica y disparar una acción inmediata hacia el entorno [2]. El flujo inicia en un estado inicial y puede terminar en uno o varios estados finales [2]. Para gestionar sistemas grandes, se utilizan superestados que contienen subestados en una estructura jerárquica, mientras que la concurrencia permite representar estados que se ejecutan en paralelo y que pueden estar sincronizados [2].

1.6.3 Diagramas de estado UML

Los diagramas de estado de UML, o máquinas de estado, son la adaptación estandarizada de los statecharts dentro del Lenguaje de Modelado Unificado [2]. En la Ingeniería de Requisitos, se emplean para especificar detalladamente el comportamiento de casos de uso o componentes críticos del sistema. UML 2 extiende estas capacidades permitiendo definir explícitamente puntos de entrada y salida en estados jerárquicos, facilitando una documentación más modular y rigurosa del comportamiento reactivo [2].

1.6.4 Aplicación a MundiPets

La teoría de los modelos de comportamiento es directamente aplicable a MundiPets para gestionar la trazabilidad de sus entidades principales. Por ejemplo, el ciclo de vida de una solicitud de adopción puede modelarse como una máquina de estados que transita de "Pendiente.^a .^{En} Revisión" tras el evento de asignación de un refugio, finalizando en .^o Probada.^o Rechazada [2]. De igual forma, el estado de una publicación de mascota podría ser "Inactiva" hasta que se cumpla la validación veterinaria (condición médica), momento en que el evento de aprobación dispara la transición al estado .^a Activa [2]. Por último, el estado de una cuenta de usuario puede reflejar comportamientos concurrentes donde la cuenta está .^a Activa para el usuario mientras el sistema, en paralelo, monitorea eventos de seguridad para transicionar a "Suspendida" si se detectan anomalías en el perfil [2].

La perspectiva de comportamiento en la ingeniería de requisitos documenta el sistema de manera orientada a los estados, capturando cómo reacciona ante eventos del contexto y qué efectos produce en su entorno [2]. Esta visión es vital para representar el comportamiento dinámico y reactivo del software, permitiendo definir no solo las transformaciones internas de los datos, sino las condiciones exactas que disparan cambios en la lógica operativa del sistema [2]. Para lograr esto, se utilizan modelos que permiten a los analistas visualizar la naturaleza reactiva del software frente a estímulos externos de forma mucho más compacta que el lenguaje natural [2].

Una de las técnicas más potentes para este fin son los statecharts, los cuales extienden la teoría de autómatas tradicionales al añadir jerarquización, condiciones de transición y concurrencia [2]. En estos modelos, un estado define un periodo donde el sistema exhibe una conducta específica mientras espera un evento para transicionar hacia un nuevo estado [2]. Los elementos clave incluyen el estado inicial, las transiciones activadas por eventos, las acciones resultantes y el uso de superestados que contienen subestados para facilitar la abstracción y el manejo de la complejidad en sistemas de gran escala [2].

En el estándar UML, estas máquinas de estado se emplean para detallar el comportamiento reactivo de los requisitos, permitiendo modelar flujos complejos con puntos de entrada y salida definidos [2]. UML adopta la jerarquía de los statecharts para permitir que los interesados comprendan rápidamente los estados de alto nivel sin perder la capacidad de profundizar en el comportamiento detallado de los sub-autómatas cuando sea necesario [2]. Esto asegura que las reglas de negocio que rigen los cambios de estado se documenten sin ambigüedades y sean verificables durante el desarrollo [2].

Para el sistema MundiPets, esta teoría permite estructurar el ciclo de vida de procesos críticos como la "Solicitud de Adopción", la cual transita por estados de revisión y aprobación condicionados a validaciones del sistema [2]. Del mismo modo, se puede modelar el estado de una "Publicación de Mascota" para que sea visible (estado .^a Activa) solo tras cumplir la condición de validación médica veterinaria [2]. Incluso la gestión de usuarios se beneficia de este enfoque, permitiendo definir estados de cuenta como "Verificada.^o "Suspendida" basados en eventos de seguridad o cumplimiento de normas, asegurando un control riguroso sobre el comportamiento del sistema ante las acciones de los actores [2].

1.7 Interrelaciones entre las tres perspectivas

En la Ingeniería de Requisitos, la documentación y el modelado se organizan habitualmente en tres perspectivas fundamentales: datos, función y comportamiento [1]. Estas perspectivas no deben considerarse como entes independientes o aislados, sino como visiones complementarias que se solapan en diversos puntos para ofrecer una representación completa del sistema [1]. El hecho de que no sean disjuntas permite a los analistas utilizar los puntos de intersección para realizar revisiones cruzadas, asegurando así que los modelos sean consistentes entre sí y que los requisitos documentados estén completos [1].

1.7.1 Relación entre perspectiva de datos y perspectiva funcional

La interrelación entre la perspectiva de datos y la funcional es crítica para entender la operación del software. Mientras que la perspectiva de datos se enfoca en la estructura estática y las dependencias de la información (mediante clases o entidades), la perspectiva funcional describe cómo esa misma información entra al sistema,

se transforma mediante procesos y finalmente sale hacia el contexto [1]. Por ejemplo, un objeto o clase definido estructuralmente en un diagrama de clases de UML se manifiesta en la perspectiva funcional como el contenido de un flujo de datos en un DFD o como un flujo de objeto en un diagrama de actividades, demostrando que las funciones del sistema dependen directamente de la estructura de los datos que manipulan [1].

1.7.2 Relación entre perspectiva funcional y perspectiva de comportamiento

La perspectiva funcional se encarga de mostrar los procesos y el movimiento de los datos, mientras que la perspectiva de comportamiento añade la dimensión dinámica al documentar cómo y cuándo cambian esos procesos en respuesta a eventos y estados específicos [1]. Existe una conexión directa entre los modelos funcionales, como los diagramas de actividades de UML o los DFD, y los diagramas de estado, ya que estos últimos definen las condiciones bajo las cuales una función puede ejecutarse [1]. En este sentido, la formalización de herramientas como los DFD incluye reglas de consistencia y balanceo que aseguran que las transformaciones funcionales sean lógicas y se alineen con el comportamiento reactivo esperado del sistema.

1.7.3 Relación entre perspectiva de datos y perspectiva de comportamiento

Existe una dependencia intrínseca entre los estados de un sistema y los datos que este almacena o modifica. En la perspectiva de comportamiento, los cambios de estado suelen ser disparados por la modificación de atributos de los datos o por condiciones lógicas que dependen de la información persistente [1]. Por ejemplo, una entidad definida en la perspectiva de datos puede tener un ciclo de vida representado en un diagrama de estados, donde su transición de un estado a otro (como pasar de una solicitud "pendiente" a "revisión") depende enteramente de la actualización de sus datos internos tras un evento del contexto [1].

1.7.4 Aplicación a MundiPets e importancia de la integración

En el sistema MundiPets, estas interrelaciones permiten gestionar procesos complejos de forma coherente. La perspectiva de datos define estructuras como "Usuario", "Mascota", "Perfil médico", las cuales son utilizadas por la perspectiva funcional en procesos como registrar mascota, "validar información médica" [1]. A su vez, la perspectiva de comportamiento rige el ciclo de vida de una "Solicitud de adopción", que transita por estados como "aceptada", "rechazada" según los eventos procesados funcionalmente [1]. Relacionar estas visiones es de suma importancia, ya que ayuda a detectar errores graves como procesos que no tienen datos de entrada, información almacenada que nunca se utiliza, o estados que no poseen eventos que activen sus transiciones, garantizando que no existan funcionalidades injustificadas en los requisitos finales [1].

La documentación de requisitos se fundamenta en la integración de tres perspectivas: datos, función y comportamiento, las cuales funcionan como visiones complementarias del sistema. Estas perspectivas no son independientes, ya que ciertos elementos de un modelo aparecen necesariamente en los otros, lo que permite verificar de manera recíproca la completitud y la consistencia de los requisitos. Mientras que la perspectiva de datos define la estructura de la información y la funcional describe su transformación a través de procesos, la perspectiva de comportamiento establece la dinámica del sistema basándose en estados y eventos que activan dichas funciones [1].

Esta interconexión se observa claramente cuando los datos definidos en modelos estructurales (como clases) se convierten en los insumos o productos de los flujos de un Diagrama de Flujo de Datos (DFD) o un diagrama de actividades [1]. En este contexto, las reglas de consistencia y el balanceo entre niveles de los modelos funcionales aseguran que la información fluya correctamente entre los procesos y los almacenes de datos persistentes [1]. Asimismo, el comportamiento reactivo documentado mediante máquinas de estado depende de los valores de estos datos para determinar cuándo se cumplen las condiciones necesarias para realizar una transición [1].

Para el sistema MundiPets, esta teoría asegura que entidades como "Usuario", "Mascota", "Perfil médico" (datos) sean correctamente manipuladas por procesos como "gestionar solicitud", "notificar resultado" (función). La dinámica de estos procesos se refleja en el ciclo de vida de las entidades, donde una solicitud de adopción cambia de estado entre "pendiente", "revisión", "finalizada" según los eventos del contexto [1]. La integración de estas tres visiones es fundamental para identificar deficiencias en el diseño, como datos que se almacenan pero no se usan, o estados que carecen de eventos que los disparen, evitando así inconsistencias en la documentación de los requisitos [1].

2 Revisión y refinamiento del SRS parcial

2.1 Registro de defectos del SRS

Tabla 1: Registro de defectos del SRS parcial.

ID	Defecto	Tipo	Corrección propuesta

2.2 Corrección con sintaxis EARS

2.3 Tabla de atributos completa (RF y RNF)

3 Modelo de datos: ER y clases UML

3.1 Identificación de entidades y relaciones

3.2 Diagrama Entidad-Relación (Crow's Foot)

3.3 Diagrama de clases UML

4 Modelo funcional: DFD y actividades UML

4.1 Diagrama de contexto (DFD Nivel 0)

En esta sección se presenta el diagrama de contexto (DFD Nivel 0) para el sistema MundiPets, el cual define los límites del sistema, sus entidades externas principales y los flujos de información generales que interactúan con el proceso central.

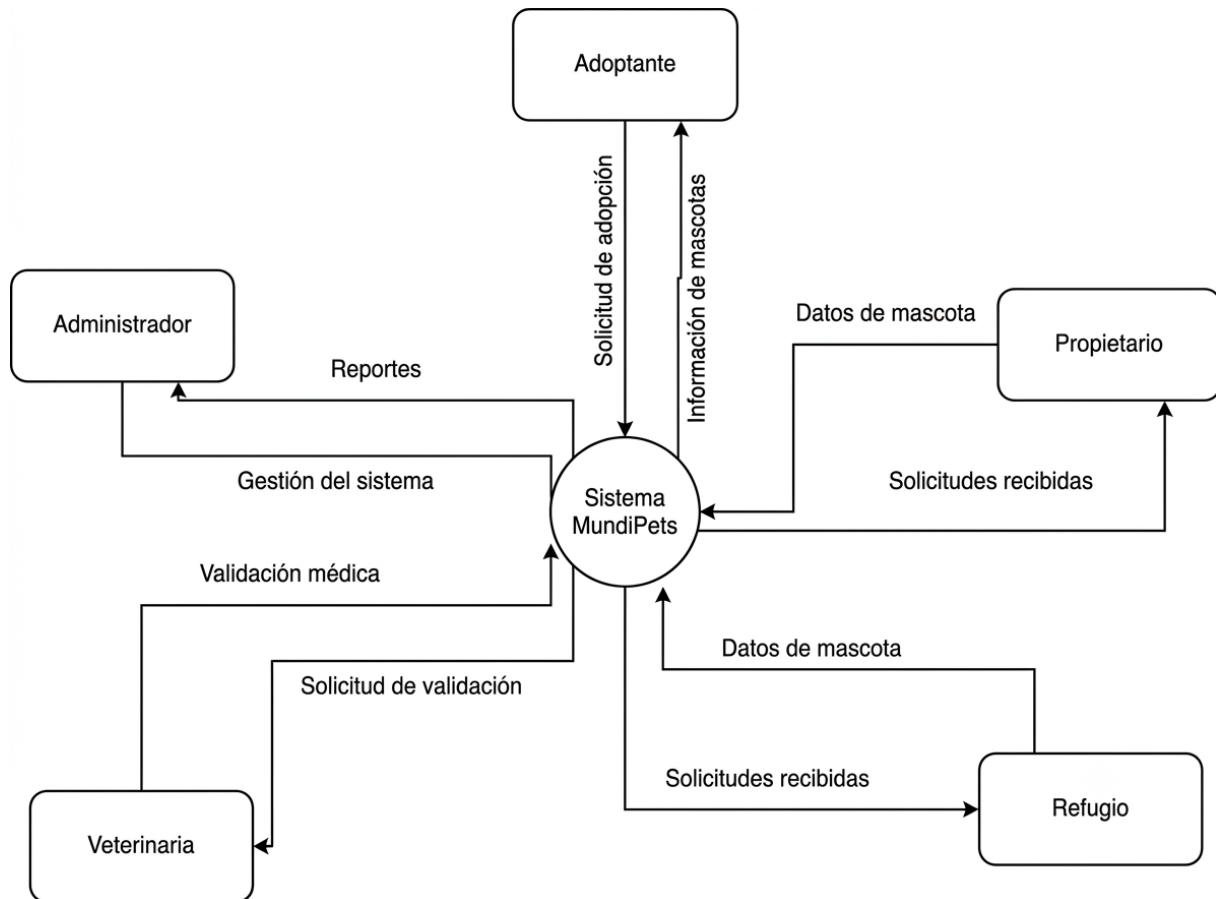


Figura 1: Diagrama de contexto (DFD Nivel 0) del sistema MundiPets.

4.2 DFD Nivel 1

El DFD de Nivel 1 descompone el proceso general del sistema MundiPets en sus procesos principales o subsistemas interactuantes, permitiendo visualizar los almacenes de datos persistentes, los flujos de datos internos y las relaciones lógicas con las entidades externas a un menor nivel de abstracción. Por su parte, el DFD esencial modela las funciones fundamentales del sistema que responden directamente a los eventos del negocio.

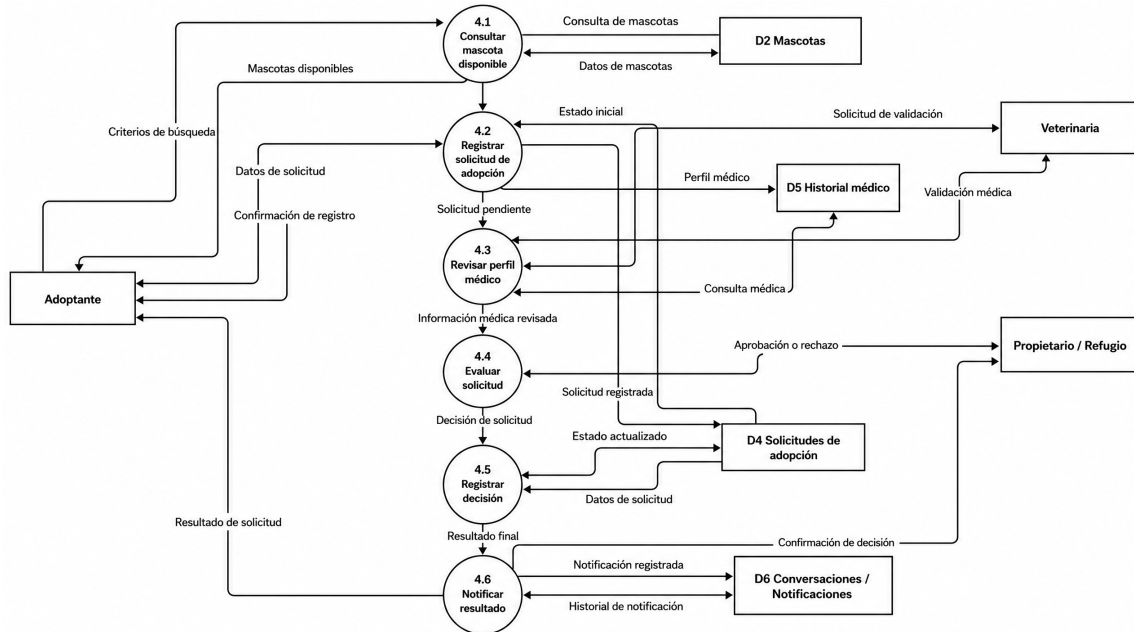


Figura 2: Diagrama de Flujo de Datos de Nivel 1 del sistema MundiPets.

4.3 DFD esencial

El DFD esencial se enfoca en capturar la lógica pura del negocio sin incluir aspectos técnicos o de implementación. Representa las respuestas fundamentales del sistema MundiPets ante los eventos externos generados por los actores, mostrando de forma directa cómo se procesan los estímulos del entorno para mantener la consistencia en el almacenamiento de la información.

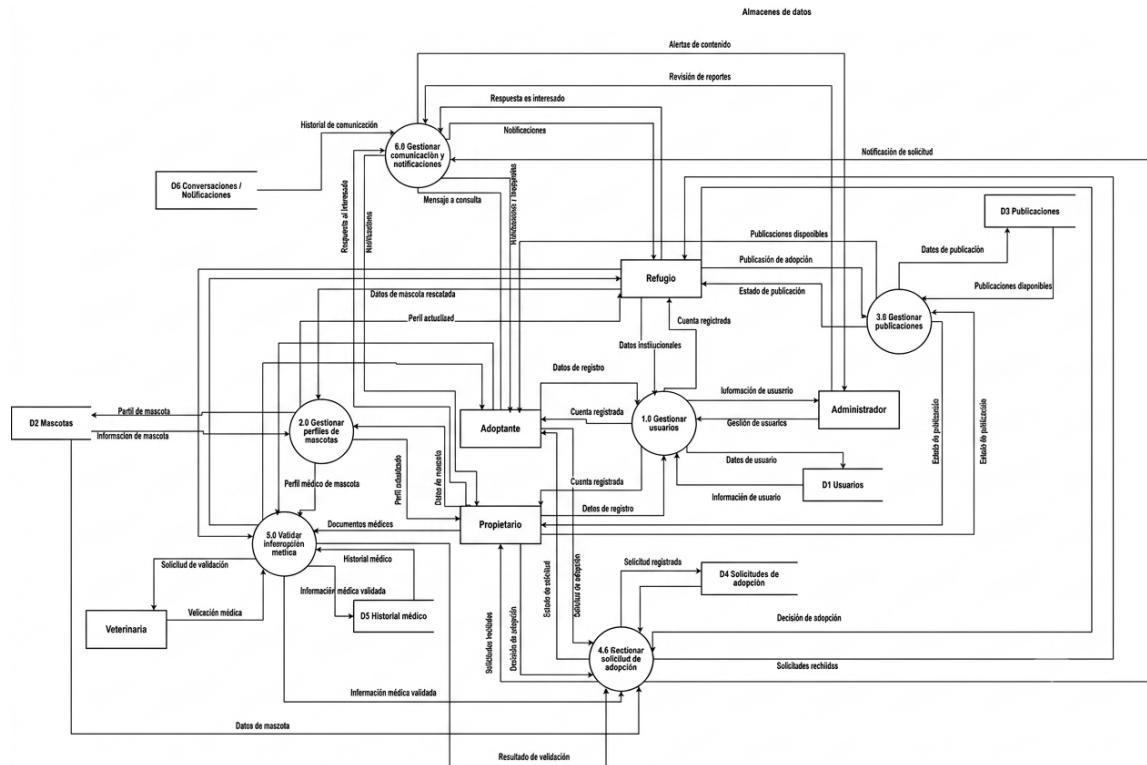


Figura 3: Diagrama de Flujo de Datos Esencial del sistema MundiPets.

4.4 Diagrama de actividades UML

El diagrama de actividades UML complementa la perspectiva funcional al describir el flujo de control, la secuencia temporal de las acciones y las condiciones lógicas que rigen los procesos de negocio críticos del sistema MundiPets. Mediante el uso de particiones o *swimlanes*, permite asignar responsabilidades específicas a cada uno de los actores involucrados (usuarios, refugios, veterinarias y el propio sistema) dentro del flujo operativo.

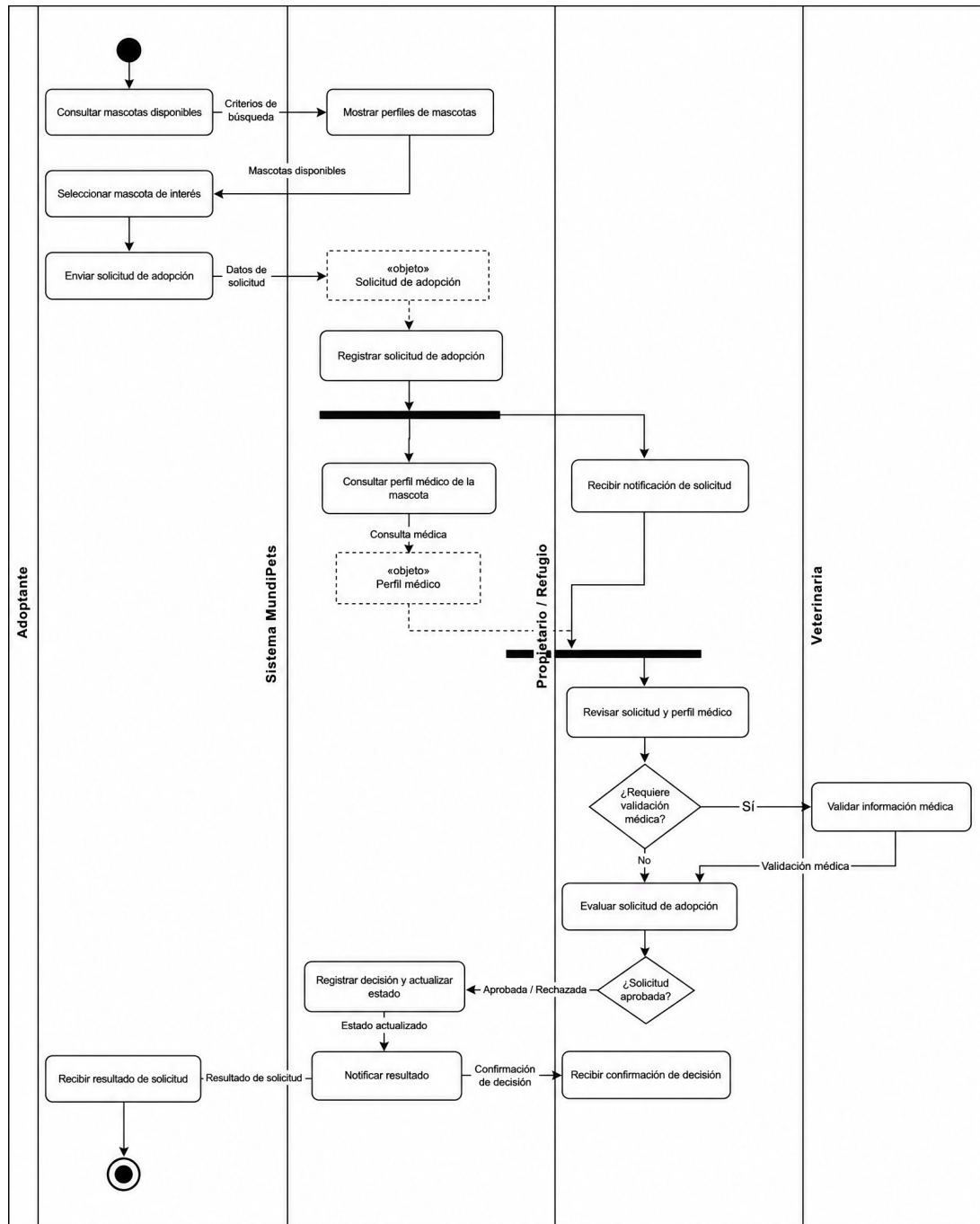


Figura 4: Diagrama de actividades UML del proceso principal en MundiPets.

5 Modelo de comportamiento: estados y secuencia

5.1 Diagrama de máquina de estados UML

5.2 Diagrama de secuencia UML

6 Matriz de trazabilidad y tabla comparativa

6.1 Matriz de trazabilidad

Tabla 2: Matriz de trazabilidad integral de componentes del sistema MundiPets.

RF	Requisito funcional	CU	Clases UML	Procesos DFD	Estados
RF-01	Registrar mascota	CU-01 Registrar mascota	Mascota, Propietario, ImagenMascota	2.0 Gestionar perfiles de mascotas / D2 Mascotas	Mascota registrada
RF-02	Gestionar historial médico de la mascota	CU-02 Registrar historial médico / CU-03 Cargar documento veterinario	HistorialMedico, DocumentoVeterinario, Mascota	5.0 Validar información médica / D5 Historial médico	Historial actualizado / pendiente de validación
RF-03	Consultar perfil completo de una mascota	CU-10 Consultar perfil médico	Mascota, HistorialMedico, Publicacion	2.0 Gestionar perfiles de mascotas / 3.0 Gestionar publicaciones	Mascota disponible / perfil consultado
RF-04	Buscar y filtrar mascotas	CU-11 Buscar mascota disponible	Mascota, Publicacion, FiltroBusqueda	3.0 Gestionar publicaciones	Publicación activa
RF-05	Gestionar solicitudes de adopción	CU-07 Gestionar proceso de adopción	SolicitudAdopcion, Adoptante, Propietario, Refugio, Mascota	4.0 Gestionar solicitud de adopción / D4 Solicitudes de adopción	Solicitud pendiente / aceptada / rechazada
RF-06	Evaluar compatibilidad entre mascotas para cruce	CU-13 Evaluar compatibilidad de cruce	Mascota, AntecedenteGenetico, CompatibilidadCruza, ModuloIA	3.0 Gestionar publicaciones / 5.0 Validar información médica	Evaluación generada / requiere revisión
RF-07	Sistema de mensajería entre usuarios	CU-08 Gestionar comunicación entre usuarios	Conversacion, Mensaje, Usuario, Notificacion	6.0 Gestionar comunicación y notificaciones / D6 Conversaciones	Mensaje enviado / leído
RF-08	Gestionar solicitudes de cruce	CU-05 Publicar mascota para cruce / CU-13 Evaluar compatibilidad de cruce	SolicitudCruza, Mascota, Propietario, CompatibilidadCruza	3.0 Gestionar publicaciones / 4.0 Gestionar solicitud	Solicitud de cruce pendiente / aceptada / rechazada
RF-09	Validar la información médica de una mascota	CU-09 Validar información médica	ValidacionMedica, Veterinaria, HistorialMedico, DocumentoVeterinario	5.0 Validar información médica / D5 Historial médico	Validación pendiente / validada / rechazada
RF-10	Gestionar recordatorios de controles preventivos	CU-12 Consultar recomendaciones de cuidado	Recordatorio, Notificacion, CarnetVacunacion, Mascota	6.0 Gestionar comunicación y notificaciones / D6 Notificaciones	Recordatorio programado / enviado
RF-11	Administrar la privacidad de la información	CU-06 Configurar privacidad médica	ConfiguracionPrivacidad, Usuario, Rol, HistorialMedico	4.0 Gestionar usuarios / 2.0 Gestionar perfiles de mascotas	Información pública / restringida
RF-12	Verificar la identidad de los usuarios	Transversal a Onboarding	Usuario, Rol, VerificacionIdentidad	1.0 Gestionar usuarios / D1 Usuarios	Usuario pendiente / verificado / rechazado

RF	Requisito funcional	CU	Clases UML	Procesos DFD	Estados
RF-13	Registrar publicaciones de mascotas	CU-04 Publicar mascota en adopción / CU-05 Publicar mascota para cruza	Publicacion, Mascota, Propietario, Refugio	3.0 Gestionar publicaciones / D3 Publicaciones	Publicación activa / inactiva
RF-14	Gestionar carnet de vacunación	CU-03 Cargar documento veterinario / CU-02 Registrar historial médico	CarnetVacunacion, Vacuna, HistorialMedico, Mascota	5.0 Validar información médica / D5 Historial médico	Carnet vigente / por vencer
RF-15	Consultar historial de procesos de adopción y cruza	CU-07 Gestionar proceso de adopción	HistorialProceso, SolicitudAdopcion, SolicitudCruza	4.0 Gestionar solicitud de adopción / D4 Solicitudes	Solicitud finalizada
RF-16	Gestionar antecedentes genéticos y parentesco de la mascota	CU-02 Registrar historial médico	AntecedenteGenetico, Mascota, HistorialMedico	5.0 Validar información médica / D5 Historial médico	Antecedente registrado / validado
RF-17	Validar imágenes mediante inteligencia artificial	Transversal a Publicaciones	ImagenMascota, ValidacionImagenIA, Publicacion, ModuloIA	3.0 Gestionar publicaciones	Imagen pendiente / validada / rechazada

6.2 Tabla comparativa de los tres modelos

Tabla 3: Cuadro comparativo de los modelos de análisis aplicados en MundiPets.

Tipo de modelo	Perspectiva que cubre	Notación usada	¿Qué requisitos ayuda a descubrir o validar?	Aplicación en MundiPets	Limitaciones
Modelo de datos	Representa la estructura de la información que el sistema necesita almacenar y relacionar. Se enfoca en entidades, atributos, relaciones y restricciones.	Diagrama Entidad-Relación con notación Crow's Foot y Diagrama de Clases UML.	Ayuda a validar requisitos relacionados con registro, consulta y almacenamiento de información. Por ejemplo: RF-01, RF-02, RF-03, RF-13, RF-14 y RF-16.	En MundiPets permite modelar entidades como Mascota, Propietario, Adoptante, Refugio, Veterinaria, Publicación, Historial Médico, Solicitud de Adopción, Solicitud de Cruza, Carnet de Vacunación y Notificación.	No muestra el orden de ejecución de los procesos ni cómo interactúan los actores paso a paso. Tampoco representa decisiones, validaciones dinámicas ni cambios de estado durante una solicitud.

Tipo de modelo	Perspectiva que cubre	Notación usada	¿Qué requisitos ayuda a descubrir o validar?	Aplicación en MundiPets	Limitaciones
Modelo funcional	Representa qué hace el sistema, qué información recibe, cómo la transforma y qué salidas produce. Se enfoca en procesos y flujos de datos.	DFD de contexto, DFD Nivel 1 y DFD esencial.	Ayuda a validar requisitos relacionados con funciones principales del sistema: búsqueda de mascotas, publicación de disponibilidad, validación médica, comunicación entre usuarios, gestión de adopciones y notificaciones.	En MundiPets se aplicó con el DFD Nivel 0, el DFD Nivel 1 y el DFD esencial. El sistema se descompone en procesos como Gestionar usuarios, Gestionar perfiles de mascotas, Gestionar publicaciones, Gestionar solicitud de adopción, Validar información médica y Gestionar comunicación y notificaciones.	No detalla la estructura interna de las clases ni los atributos de cada entidad. También puede mostrar el flujo de datos, pero no siempre explica con claridad el orden temporal completo ni los estados por los que pasa una solicitud.
Modelo de comportamiento	Representa cómo se comporta el sistema ante eventos, decisiones y cambios de estado. Se enfoca en la secuencia de acciones, condiciones y respuestas.	Diagrama de actividades UML, swimlanes, nodos de decisión, fork/join, flujos de objeto y estados.	Ayuda a validar requisitos donde importa el flujo paso a paso. Por ejemplo: RF-05 Gestionar solicitudes de adopción, RF-07 Sistema de mensajería, RF-09 Validar información médica, RF-10 Gestionar recordatorios y RF-15 Consultar historial de procesos.	En MundiPets se aplicó al flujo Gestionar solicitud de adopción, donde el adoptante consulta mascotas, registra una solicitud, el sistema revisa el perfil médico, el propietario o refugio evalúa la solicitud, se registra la decisión y se notifica el resultado.	Normalmente se enfoca en un proceso específico, por lo que no cubre todos los requisitos funcionales del sistema. Además, no define tablas, clases ni almacenamiento de datos; para eso se necesita el modelo de datos.

6.3 Conclusiones

En MundiPets, esta perspectiva ayuda a validar requisitos como el registro de mascotas, historial médico, publicaciones, carnet de vacunación, solicitudes y antecedentes genéticos. Sin este modelo, sería difícil verificar si entidades como Mascota, Usuario, Historial Médico, Publicación, Solicitud de Adopción o Notificación tienen relación directa con los requisitos funcionales. Por ello, el modelo de datos sirve como base para asegurar que el sistema almacene la información necesaria para los procesos de adopción, cruza y validación médica.

La perspectiva funcional permite identificar cómo se transforman los datos dentro del sistema. Los DFD permiten observar qué información entra, qué procesos la utilizan y qué salidas genera el sistema. En MundiPets, esta perspectiva cubre procesos como gestionar usuarios, gestionar perfiles de mascotas, gestionar publicaciones, gestionar solicitudes de adopción, validar información médica y gestionar comunicación/notificaciones. Esto se relaciona con las funciones principales de la plataforma, como búsqueda de mascotas, publicación de disponibilidad, validación médica y comunicación entre usuarios.

La perspectiva de comportamiento permite entender el orden de las acciones y decisiones dentro de un caso de uso. El diagrama de actividades UML complementa al DFD porque no solo muestra flujos de datos, sino también la secuencia del proceso. En MundiPets, el flujo de gestionar solicitud de adopción permite representar cómo el adoptante consulta mascotas, registra una solicitud, el sistema revisa el perfil médico, el propietario

o refugio evalúa la solicitud, se registra la decisión y finalmente se notifica el resultado. Esto ayuda a validar RF-05, RF-09, RF-07 y RF-15, que dependen de estados, decisiones y notificaciones.

Los gaps parciales muestran que algunos requisitos necesitan reforzarse en los modelos. Aunque no se encontraron requisitos funcionales totalmente sin cobertura, sí se detectaron gaps parciales en RF-06, RF-08, RF-10 y RF-17. Estos requisitos están relacionados con compatibilidad de cruza, solicitudes de cruza, recordatorios preventivos y validación de imágenes mediante inteligencia artificial.

Para mejorar la consistencia del modelo, se recomienda representar con mayor claridad estos elementos en los DFD, diagramas de actividades o estados. Esto es importante porque MundiPets incluye apoyo inteligente para compatibilidad de cruza y validación de imágenes, pero esas funciones deben quedar explícitas en los modelos para evitar ambigüedad.

La revisión de gold plating ayuda a mantener el sistema dentro de su alcance real. La comparación entre requisitos y modelos permite evitar funcionalidades que no están justificadas por los RF. En MundiPets, elementos como pagos en línea, transporte de mascotas, diagnóstico veterinario automático, gestión clínica completa o red social general no deben agregarse como funciones principales, porque están fuera del alcance definido para el sistema. La ERS indica que MundiPets no reemplaza consultas veterinarias, no gestiona clínicas, no maneja pagos ni transporte, y tampoco funciona como una red social general.

En conclusión, las tres perspectivas de especificación se complementan para representar de manera más completa el sistema MundiPets. El modelo de datos permite identificar la información que debe almacenarse; el modelo funcional permite visualizar los procesos y flujos de datos; y el modelo de comportamiento permite comprender la secuencia de acciones, decisiones y estados. La matriz de trazabilidad demuestra que los requisitos funcionales tienen cobertura general, pero también evidencia la necesidad de reforzar algunos requisitos vinculados con cruza, recordatorios e inteligencia artificial para mejorar la consistencia entre los modelos.

Anexo A – Tabla de atributos completa de todos los RF y RNF

Anexo B – Registro de defectos del SRS con correcciones aplicadas

Anexo C – Exportaciones originales de todos los diagramas (alta resolución)

Anexo D – Referencias IEEE y declaración de uso de IA

Referencias

- [1] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, 2025.
- [2] R. Ibrahim y S. Y. Yen, “Formalization of the Data Flow Diagram Rules for Consistency Check,” *International Journal of Software Engineering & Applications*, vol. 1, n.º 4, págs. 95-111, oct. de 2010. DOI: 10.5121/ijsea.2010.1406